

Reviewer Pack — Minimal Rank of Quadratic Forms vs Communication Complexity ...

Entry 324cadb0fb5c · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-23 05:56:43 UTC

Minimal Rank of Quadratic Forms vs Communication Complexity for k-CNF

Entry ID: 324cadb0fb5c **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-23 05:56:43 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Quadratic Forms) **Field B** (complexity object): Communication Complexity: k-CNF

Statement:

[‘For all instances of k-CNF, the communication complexity is upper-bounded by a function of the minimal rank of the quadratic form associated with the CNF.’, ‘For any instance of size n , there exists a quadratic form such that its minimal rank is $O(n^k)$, where k is a constant, and for this quadratic form, the communication complexity for k-CNF is also $O(n^k)$.’, ‘No instance of size n requires more than $O(n^{k+1})$ communication for k-CNF, where k is the minimal rank of the associated quadratic form.’]

Rationale (proposer’s reasoning):

[‘Quadratic forms provide a structured way to encode boolean functions, which could be related to communication complexity. If the minimal rank of the quadratic form is high, it suggests a complex function that requires more communication to describe.’, ‘The relationship between the structure of quadratic forms and communication complexity in k-CNF instances might reveal new insights into the computational power of communication protocols.’, ‘This conjecture aims to establish a connection between algebraic structures and complexity measures, potentially leading to better understanding of communication complexity lower bounds.’]

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2f65ac30e478b240

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all k-CNF instances with n variables ($n \leq 40$), the communication complexity is $O(n^k)$ and the minimal rank of the quadratic form associated

with the CNF is also $O(n^k)$. The conjecture is falsified if any instance requires communication complexity greater than $O(n^{(k+1)})$ or a quadratic form with minimal rank less than $O(n^k)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quadratic forms" AND "communication complexity" AND k-CNF" - "minimal rank" IN quadratic forms AND $O(n^k)$ AND communication complexity" - "algebraic combinatorics" AND minimal rank of quadratic forms AND $O(n^{(k+1)})$ communication complexity

Top relevant hits considered: - [http://arxiv.org/abs/2605.05444v1] Searches for Binary Mergers with Sub-solar Mass Components in Data from the First Part of LIGO-Virgo-KAGRA's Fourth Ob - [http://arxiv.org/abs/2011.11054v2] Consecutive Quadratic Residues And Quadratic Nonresidue Modulo p - [http://arxiv.org/abs/2412.14709v1] Minimal rank of primitively n -universal integral quadratic forms over local rings - [http://arxiv.org/abs/1810.10361v2] Computational complexity, Newton polytopes, and Schubert polynomials - [http://arxiv.org/abs/1711.02729v2] On f - and h - vectors of relative simplicial complexes - [http://arxiv.org/abs/1809.01026v1] Toric degenerations of Grassmannians from matching fields - [s2:1401.3467] Planning over Chain Causal Graphs for Variables with Domains of Size 5 Is NP-Hard

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(n, k):
        clauses = []
        for _ in range(k * n):
            clause = [random.choice([-1, 1]) * (i + 1) for i in range(n)]
            if all(clause[i] == -clause[j] for i in range(n) for j in range(i + 1, n)):
                continue
            clauses.append(clause)
        return clauses

    def quadratic_form(cnf):
```

```

n = len(cnf[0])
Q = [[0] * n for _ in range(n)]
for clause in cnf:
    for i in range(n):
        for j in range(i, n):
            Q[i][j] += clause[i] * clause[j]
            if i != j:
                Q[j][i] = Q[i][j]

return Q

def min_rank(matrix):
    n = len(matrix)
    rank = 0
    for i in range(n):
        pivot = matrix[i][i]
        if pivot == 0:
            continue
        rank += 1
        for j in range(i + 1, n):
            matrix[j][i] /= pivot
        for j in range(n):
            if j != i:
                factor = matrix[j][i]
                for k in range(n):
                    matrix[j][k] -= factor * matrix[i][k]

    return rank

def communication_complexity(rank):
    return rank ** 2

n = random.randint(5, 40)
k = random.randint(1, 3)
cnf = generate_k_cnf(n, k)
Q = quadratic_form(cnf)
rank = min_rank(Q)
comm_complexity = communication_complexity(rank)

return {
    "metric_name": "communication_complexity",
    "metric_value": comm_complexity,
    "instances_tested": 1,
    "conjecture_holds": comm_complexity <= n ** k and rank == n ** k,
    "counterexample": f"rank={rank}, expected=k={k}" if not (comm_complexity <= n ** k and rank == n ** k)
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_comm_complexity = sum(r["metric_value"] for r in results) / len(results)

```

```

std_comm_complexity = math.sqrt(sum((r["metric_value"] - mean_comm_complexity) ** 2 for r in results))
support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

if all(r["conjecture_holds"] for r in results):
    print(f"RESULT: SUPPORTED mean={mean_comm_complexity} std={std_comm_complexity} support_fraction={support_fraction}")
elif any(not r["conjecture_holds"] for r in results):
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{r['counterexample']}\" first_failing_seed={first_failing_seed}")
else:
    print("RESULT: INCONCLUSIVE")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

plexity', 'metric_value': 49, 'instances_tested': 1, 'conjecture_holds': False, 'counterexample': 'r
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 625, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 529, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 81, 'instances_tested': 1, 'conje
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 1444, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 256, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 1296, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 361, 'instances_tested': 1, 'con
TRIAL: {'metric_name': 'communication_complexity', 'metric_value': 289, 'instances_tested': 1, 'con

```

--STDERR--

Traceback (most recent call last):

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44a0d8ce.py", line 93, in <module>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^

```

```

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44a0d8ce.py", line 93, in <genexpr>
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
                        ~~~~~^

```

KeyError: 'seed'

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a definitive evaluation of the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15723
2	propose	ollama_remote	glm4:latest	0	0	13370
3	preregistration	ollama_remote	glm4:latest	0	0	9668
4	novelty	ollama_remote	glm4:latest	0	0	11046
5	novelty	ollama_remote	glm4:latest	0	0	9773
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15392
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9925
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9297
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9729
10	judge	ollama_remote	glm4:latest	0	0	14101

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 118024 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/324cadb0fb5c.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/324cadb0fb5c.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/324cadb0fb5c.tar.gz` (if generated)